

DeltaNet Variant Investigation Report

n/r = not reported in accessible paper/model-card text.

Sources: [DeltaNet \(Schlag et al., 2021\)](#), [Gated DeltaNet \(Yang et al., 2024\)](#), [GLA / Flash Linear Attention \(Yang et al., 2024\)](#), [RLA / RDN \(Lai et al., 2025\)](#), [Kimi Linear \(Kimi Team, 2025\)](#), [GLA 1.3B model card](#), [GLA 340M model card](#), [NVLabs GatedDeltaNet repo](#), [Kimi Linear repo](#).

Table 1a — Pretraining Datasets

Dataset	Tokens	Used in	In repo?	Integration	Notes
SlimPajama	15B / 100B subsets	GLA (15B for 340M, 100B for 1.3B); RLA/RDN (100B)	No	EASY	Standard Megatron d path fits it; recommender primary dataset for reproducibility since t GLA and RLA/RDN u
FineWeb-Edu	15B / 100B subsets	Gated DeltaNet (100B); GDN ablations (15B)	No	EASY	Used for all GDN experiments; Megatron generic preprocessing support
WikiText-103	~103M words	DeltaNet (train + eval); RLA/RDN (eval)	Partly	EASY	Megatron examples/c reference it; tokenizer/preprocess exists
K2 Corpus	1.4T / 5.7T	Kimi Linear	No	N/A	Proprietary Moonshot corpus; not publicly available
Nemotron ClimbMix (climbmix_small)	~6–12B	Repo only (Gipfelsturm)	Yes	INCLUDED	Already converted to Megatron .bin/.ids useful for quick smok
WMT14 En-De	n/r in tokens	DeltaNet (MT experiments)	Partly	HARD	Repo has preprocess_data_n but launcher is GPT-pretrain oriented

Table 1b — Evaluation Benchmarks

Benchmark	Type	Used in	In repo?	Integration	Notes
WikiText-103 perplexity	LM perplexity	DeltaNet; GDN; GLA; RLA/RDN	Partly	EASY	Standard LM eval; cheap and fast
LAMBADA (LMB)	LM / zero-shot	GDN; RLA/RDN	No	EASY	Via <code>lm-eval</code> harness
Commonsense suite (PIQA, HellaSwag, WinoGrande, ARC-e, ARC-c, SIQA, BoolQ)	Zero-shot reasoning	GDN; RLA/RDN	No	MEDIUM	Requires <code>lm-eval</code> harness install + checkpoint export
MMLU	Few-shot knowledge	RLA/RDN; Kimi Linear	Partly	EASY	ModelOpt MMLU script exists using <code>cais/mmlu</code> from HF
MMLU-Pro	Few-shot knowledge	Kimi Linear	No	MEDIUM	Via <code>lm-eval</code> harness
Recall suite (SWDE, SQuAD, FDA, TriviaQA, NQ, DROP, NIAH)	In-context recall	GDN; RLA/RDN	No	HARD	Custom eval scripts needed; measures associative recall ability
LongBench / LongBench V2	Long-context QA	GDN; Kimi Linear	No	HARD	14 tasks covering single/multi-doc QA, summarization, few-shot, code
RULER	Long-context synthetic	GDN; Kimi Linear	No	HARD	Needle-in-a-haystack style; external NVIDIA/RULER tooling needed
S-NIAH	Long-context synthetic	GDN	No	HARD	Single-needle-in-a-haystack variant
HELMET-ICL	Long-context ICL	Kimi Linear	No	HARD	In-context learning benchmark

Benchmark	Type	Used in	In repo?	Integration	Notes
Synthetic tasks (Palindrome, MQAR, Stack)	Synthetic reasoning	Kimi Linear	No	MEDIUM	Tests state tracking and associative recall; small custom setups
GSM8k, HumanEval	Math / code	RLA/RDN (ablation)	No	MEDIUM	Standard math and code benchmarks
MATH 500, AIME 2025	Math reasoning	Kimi Linear (RL)	No	HARD	Used for RL training evaluation

Table 2 — Pretraining Configurations from Literature

DeltaNet

Config	Params	Layers	Hidden	Heads	FFN	Seq len	Tokens	Dataset
Small	40M	16	128	8	2048 (16x)	n/r	~103M words	WikiText-103
Medium	90M	16	256	n/r	n/r	n/r	~103M words	WikiText-103

GLA (Flash Linear Attention)

From HuggingFace model cards (`config.json`):

Config	Params	Layers	Hidden	Heads	Seq len	Tokens	Dataset
340M	340M	24	1024	4 (default but did different abeltations)	n/r	15B	SlimPajama
1.3B	1.3B	24	2048	4 (default but did different abeltations)	n/r	100B	SlimPajama

Activation: SwiSH. Output gate: yes. Gated key: yes. Gated value: no. Vocab: 32,000 (LLaMA-2 tokenizer).

Gated DeltaNet

The paper uses a LLaMA-style macro architecture but does not publish an explicit layer/dimension table. Known details:

Config	Params	Tokens	Dataset	Head dim	Seq len	Notes
Pure recurrent	400M	100B	FineWeb-Edu	128 (best from ablation)	4096	SwiGLU MLP, L2-norm on Q/K
Pure recurrent	1.3B	100B	FineWeb-Edu	128	4096	Same setup
Hybrid H1	1.3B	100B	FineWeb-Edu	128	4096	GDN + SWA (2K window)
Hybrid H2	1.3B	100B	FineWeb-Edu	128	4096	Mamba2 + GDN + SWA (2K window)

RLA / Residual DeltaNet

From paper Table 6 and Table 7:

Config	Params	Layers	Hidden	Intermediate	Heads	GQA groups	
Transformer baseline	1.51B	16	2048	8192	32	4	
Linear attention (RLA/RDN/sGLA/GDN etc.)	1.55B	16	2048	8192	16	16	

All trained on 100B tokens from SlimPajama. Tokenizer: n/r.

Kimi Linear

All a bit more complex also due to MoE and different releases etc.

Config	Total params	Activated params	KDA:MLA ratio	Seq len (pretrain)	Seq len (final)	Tokens	Dataset
Fair comparison	48B	3B	3:1	4096	—	1.4T	K2 corpus
Released checkpoint	48B	3B	3:1	4096	1M	5.7T	K2 corpus

Scaling law experiment configs (from paper Table 2):

Activated params	Heads	Layers	Hidden	Tokens	LR	Batch size
653M	16	16	1216	38.8B	2.006e-3	336
878M	18	18	1376	59.8B	1.790e-3	432
1.1B	20	20	1536	85.2B	1.617e-3	512
1.4B	22	22	1632	102.5B	1.486e-3	576
1.7B	24	24	1776	128.0B	1.371e-3	640

All scaling law models use context length 4096. KDA extends GDN by replacing the scalar forget gate with a channel-wise (per-dimension) forget gate vector.

Table 3 — Optimizer and Training Hyperparameters

Paper	Optimizer	Peak LR	Min LR	WD	Betas	Batch (tokens)	Warmup	Schedule
DeltaNet	Adam	2.5e-4	n/r	n/r	default	seq batch 96 (small) / 56 (medium)	2000 steps	n/r
GLA	AdamW	3e-4	3e-5	0.01	n/r	0.5M (340M) / 2M (1.3B)	0.5B / 1B tokens	Cosine
Gated DeltaNet	AdamW	4e-4	n/r	0.1	n/r	0.5M	1B tokens	Cosine
RLA / RDN	AdamW	3e-4	3e-5	0.1	n/r	4M	0.5B tokens	Cosine
Kimi Linear	MuonClip	1.1e-3	n/r	n/r	n/r	32M	n/r	WSD

Notes:

- DeltaNet also includes synthetic associative-retrieval tasks and WMT14 En-De MT; the row above uses the WikiText-103 LM setup.
- Gated DeltaNet uses the LLaMA-2 tokenizer (32K vocab).
- GLA uses initial and final LR of 3e-5.

- RLA/RDN initializes weights from a normal distribution with constant std 0.006.
- Kimi Linear uses the WSD (Warmup-Stable-Decay) schedule and the MuonClip optimizer (a variant of Muon with clipped updates). The Muon optimizer achieves ~2x computational efficiency vs AdamW at scale (demonstrated by Moonlight/Moonshot). All Kimi scaling law models share the same annealing and long-context activation phase from Kimi K2.
- Hardware: DeltaNet explicitly mentions 3x V100 (WMT14) and 2x V100 (WikiText LM). GDN throughput benchmarks use a single H100. Other papers underreport hardware.

Evaluation Results Screenshots

DeltaNet (2021)

Table 1 — WMT14 En-De BLEU scores:

*Table 1. WMT14 En-De Translation BLEU scores for various Transformer models. Neither model averaging, nor model specific tuning is done. **Standard** denotes the basic Transformer.*

d_{dot}	Valid			Test		
	64	256	512	64	256	512
Standard	26.6	-	-	27.7	-	-
Linear	25.5	-	-	26.8	-	-
Performer	24.2	24.9	26.7	24.4	25.3	27.7
DPFP (ours)	-	26.2	26.2	-	26.9	27.1

Table 2 — WikiText-103 perplexity (small: 40M, medium: 90M):

Table 2. WikiText-103 language model perplexity results showing effects of our update rule. The number of trainable parameters are almost the same for all models, up to the small difference introduced by gating in our update rule (16 K and 33 K parameters respectively for the *small* and *medium* configurations). We have $D = 128$, $L = 256$ (40 M parameters) in the *small*, and $D = 256$, $L = 384$ (90 M parameters) in the *medium* configuration. For Performers, m is 8 and 16, respectively.

	Update	<i>small</i>		<i>medium</i>	
	Rule	Valid	Test	Valid	Test
Transformer	-	33.0	34.1	27.9	29.6
Linear Transformer	sum	37.1	38.3	31.1	33.0
Delta Network	delta	34.1	35.5	29.7	31.5
Performer	sum	39.0	39.6	32.2	33.8
	delta	36.1	37.2	30.0	31.8

Table 3 — WikiText-103 perplexity ablation (position encoding / attention normalization):

Table 3. WikiText-103 language model perplexities for Linear Transformers (*medium* configuration) with our update rule.

Position Encoding	Attn. Normalisation	Valid	Test
Yes	Yes	30.4	32.1
No	Yes	29.2	31.2
Yes	No	29.7	31.5
No	No	28.1	31.1

Table 4 — WikiText-103 perplexity without truncating context (vs Transformer-XL):

Table 4. WikiText-103 language model perplexities when the model is trained and evaluated **without truncating context**, as opposed to Table 2 where the context window is limited. The *medium* config is used. Neither positional encoding nor attention normalisation is used for the Delta Net. The numbers of trainable parameters (Prms.) are given in millions. We compare with the Transformer-XL at different memory segment lengths. This results in different state sizes which are proportional to the memory requirements during evaluation, and highlights the memory efficiency of the Delta Network. The state sizes are given in millions.

Model	Prms. in M.	State size in M.	Perplexity	
			Valid	Test
Linear Transformer	89.8	0.13	>260	>260
Delta Network	89.9	0.13	27.8	29.4
Transformer-XL	90.9	0.13	65.7	65.5
		1.05	29.3	30.1
		2.10	26.4	27.4
		6.29	24.6	25.5

Table 1 shows the BLEU score (Papineni et al., 2002; Post, 2018) results. The Performer is as good as the basic Transformer when the number of samples m is large enough (for $d_{\text{dot}} = 512$, we have $m = 256$). In fact, with $d_{\text{key}} = 64$, the recommended value for m is $d_{\text{dot}} \log(d_{\text{dot}}) = 266$. Our DPFP model outperforms the Linear Transformer as well as the Performer when d_{dot} is relatively small; providing a good trade-off between simplicity and performance.

Gated DeltaNet (2024)

Table 3 — Language modeling and zero-shot commonsense reasoning (1.3B, 100B tokens, FineWeb-Edu):

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	BoolQ acc ↑	Avg.
<i>Recurrent models</i>											
RetNet	19.08	17.27	40.52	70.07	49.16	54.14	67.34	33.78	40.78	<u>60.39</u>	52.02
HGRN2	19.10	17.69	39.54	70.45	49.53	52.80	69.40	35.32	<u>40.63</u>	56.66	51.79
Mamba	17.92	15.06	43.98	71.32	52.91	52.95	69.52	35.40	37.76	61.13	53.12
Mamba2	<u>16.56</u>	<u>12.56</u>	<u>45.66</u>	<u>71.87</u>	<u>55.67</u>	<u>55.24</u>	72.47	<u>37.88</u>	40.20	60.13	<u>54.89</u>
DeltaNet	17.71	16.88	42.46	70.72	50.93	53.35	68.47	35.66	40.22	55.29	52.14
Gated DeltaNet	16.42	12.17	46.65	72.25	55.76	57.45	<u>71.21</u>	38.39	<u>40.63</u>	60.24	55.32
<i>Attention or hybrid models</i>											
Transformer++	18.53	18.32	42.60	70.02	50.23	53.51	68.83	35.10	40.66	57.09	52.25
Samba	16.13	13.29	44.94	70.94	53.42	55.56	68.81	36.17	39.96	<u>62.11</u>	54.00
Gated DeltaNet-H1	<u>16.07</u>	12.12	<u>47.73</u>	72.57	<u>56.53</u>	58.40	<u>71.75</u>	40.10	<u>41.40</u>	63.21	56.40
Gated DeltaNet-H2	15.91	<u>12.55</u>	48.76	<u>72.19</u>	56.88	<u>57.77</u>	<u>71.33</u>	<u>39.07</u>	41.91	61.55	<u>56.18</u>

Table 3: Performance comparison on language modeling and zero-shot common-sense reasoning.

Table 4 — Recall-world retrieval tasks (input truncated to 2K tokens):

Models	SWDE	SQD	FDA	TQA	NQ	Drop	Avg
<i>Recurrent models</i>							
RetNet	14.0	28.5	7.0	54.4	16.2	17.3	22.9
HGRN2	8.3	25.3	4.8	51.2	14.2	16.9	20.1
Mamba	9.8	25.8	3.7	54.3	14.9	17.4	21.0
Mamba2	<u>19.1</u>	<u>33.6</u>	25.3	61.0	20.8	<u>19.2</u>	<u>29.8</u>
DeltaNet	17.9	30.9	18.4	53.9	17.3	18.6	26.2
Gated DeltaNet	25.4	34.8	<u>23.7</u>	<u>60.0</u>	<u>20.0</u>	19.8	30.6
<i>Attention or hybrid models</i>							
Transformer++	29.5	38.0	52.2	58.3	22.5	21.6	37.0
Samba	33.0	39.2	50.5	57.7	23.5	20.2	37.3
Gated DeltaNet-H1	<u>35.6</u>	<u>39.7</u>	<u>52.0</u>	<u>60.1</u>	<u>24.6</u>	<u>22.2</u>	<u>39.0</u>
Gated DeltaNet-H2	38.2	40.4	50.7	63.3	24.8	23.3	40.1

Table 4: Accuracy on recall-world retrieval tasks with input truncated to 2K tokens. SQD: SQUADE. TQA: Trivial QA.

Figure 2 — Length extrapolation on six long-context benchmarks:

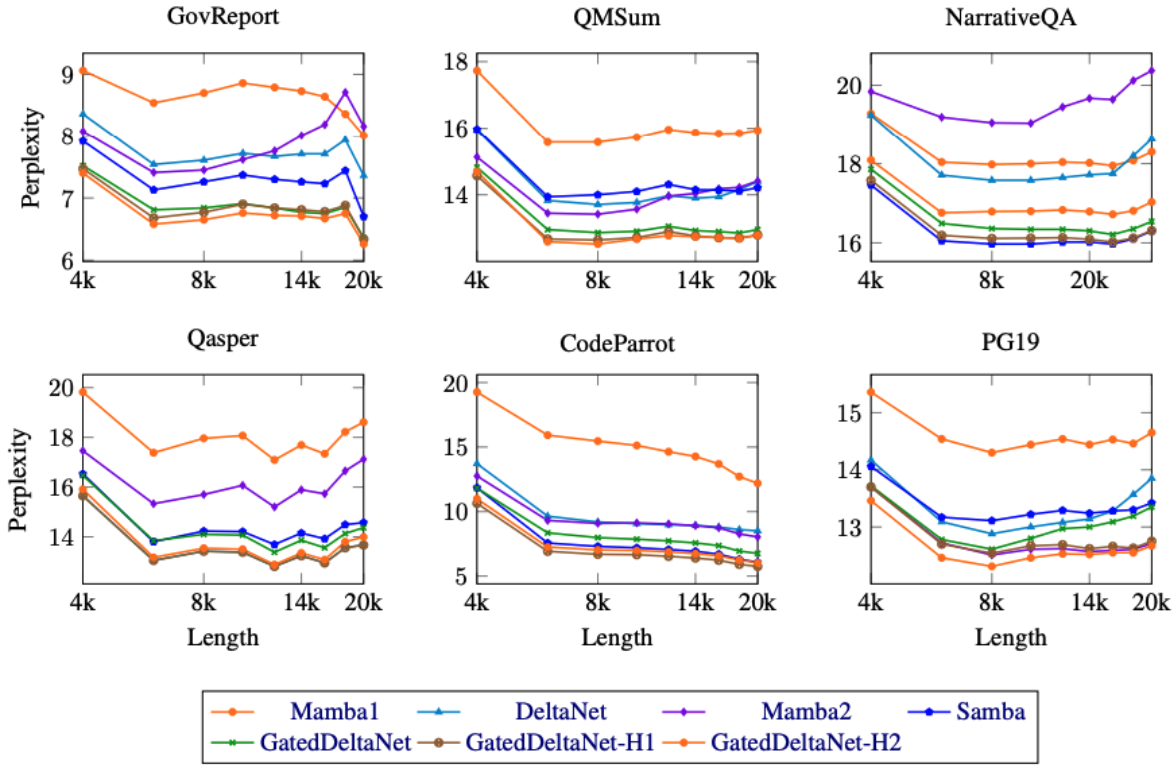


Figure 2: Length extrapolation on six long benchmarks.

Length extrapolation on long sequences. As shown in Fig. 2, we evaluate the models’ capacity to extrapolate to sequences of up to 20K tokens across six long-context benchmarks. Gated DeltaNet achieves the lowest overall perplexity across tasks among RNN models. While we observe mixed results in length extrapolation, Gated DeltaNet exhibits relatively more robust performance, suggesting better memory management. The hybrid models further improve upon this by leveraging attention for local context modeling, which reduces the memory management burden on their recurrent components. Future work will explore these models’ capabilities on even longer sequences.

Table 5 — LongBench accuracy (14 tasks):

Model	Single-Doc QA			Multi-Doc QA			Summarization			Few-shot			Code		Avg
	NQA	QQA	MFQ	HQA	2WM	Mus	GvR	QMS	MNs	TRC	TQA	SSM	LCC	RBP	
Recurrent models															
RetNet	12.1	10.7	19.1	10.7	18.0	5.8	4.8	15.8	7.9	19.0	18.0	<u>12.8</u>	14.1	17.9	13.2
HGRN2	10.7	<u>12.1</u>	19.1	11.3	15.7	<u>6.0</u>	5.2	15.1	9.2	16.0	15.8	10.3	<u>18.6</u>	<u>20.8</u>	13.5
Mamba	<u>13.0</u>	10.1	20.4	10.1	<u>16.7</u>	<u>6.0</u>	<u>7.2</u>	<u>15.9</u>	<u>8.4</u>	<u>23.1</u>	21.9	11.2	17.9	19.0	<u>14.6</u>
DeltaNet	12.9	10.8	<u>21.5</u>	<u>10.9</u>	13.2	5.1	6.5	13.5	7.2	15.5	<u>23.3</u>	11.6	17.6	20.3	13.6
Mamba2	11.1	11.3	18.6	11.8	15.1	6.7	6.7	14.5	7.4	13.0	23.6	8.4	17.9	20.6	13.5
Gated DeltaNet	14.1	14.0	23.3	13.7	14.4	5.8	7.5	16.4	7.9	30.0	22.4	23.0	18.7	22.1	16.6
Attention or hybrid models															
Transformer++	11.8	9.3	10.0	10.9	4.2	6.1	7.4	15.8	6.6	16.9	13.5	3.9	17.2	18.7	11.0
Samba	12.5	<u>12.9</u>	25.4	11.2	19.7	<u>6.8</u>	<u>9.1</u>	15.7	11.0	20.0	<u>22.7</u>	22.8	<u>18.1</u>	<u>21.1</u>	<u>15.9</u>
Gated DeltaNet-H1	14.5	12.3	<u>26.6</u>	<u>12.6</u>	23.6	6.1	<u>9.1</u>	<u>16.1</u>	<u>12.8</u>	<u>33.5</u>	23.9	<u>26.8</u>	15.5	19.2	17.8
Gated DeltaNet-H2	<u>12.7</u>	13.0	27.1	<u>12.7</u>	<u>20.6</u>	7.5	10.4	16.2	13.0	40.5	<u>22.7</u>	27.9	19.9	22.1	18.4

Table 5: Accuracy on 14 tasks from LongBench (Bai et al., 2023): Narrative QA, QasperQA, MultiField QA, HotpotQA, 2WikiMulti QA, Musique, GovReport, QMSum, MultiNews, TRec, Trivia QA, SamSum, LCC, and RepoBench-P by order.

Figure 3 — Training throughput comparison (1.3B, single H100):

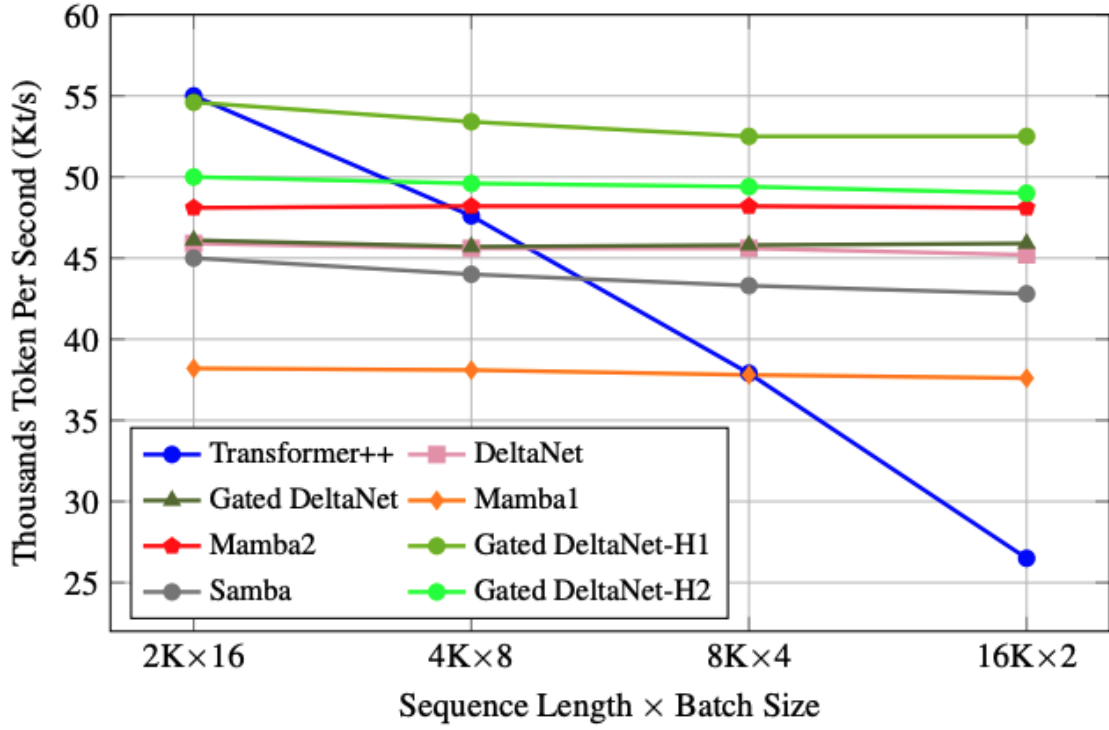
**Figure 3:** Training throughput comparison of 1.3B models on a single H100 GPU.

Table S.1 — GDN ablation study (400M, 15B tokens): macro design, normalization, feature maps, head dimensions:

Table S.1: Ablation study on the Gated DeltaNet block. Avg-PPL and Avg-Acc denote average perplexity and zero-shot commonsense reasoning accuracy (as in Table 3), respectively. All models have 400M parameters and are trained for 15B tokens on the same subset of FineWeb-Edu dataset (Penedo et al., 2024).

<i>Gated DeltaNet Ablations (400M)</i>	Avg-PPL (↓)	Avg-Acc (↑)
Gated DeltaNet w Head Dim 128,	27.35	47.26
<i>Macro Design</i>		
w. naive Delta Rule	30.87	45.12
w/o. Short Conv	28.95	46.16
w/o. Output Gate	29.12	45.46
w/o. Output Norm	27.55	47.07
<i>Normalization & Feature Map</i>		
w. L_1 -norm & ReLU	30.79	45.92
w. L_1 -norm & 1+ELU	30.34	46.05
w. L_1 -norm & SiLU	30.18	46.09
w. L_2 -norm & ReLU	27.67	46.94
w. L_2 -norm & 1+ELU	27.58	47.17
<i>Model Dimensions</i>		
w. Head Dim 64	28.31	46.35
w. Head Dim 256	27.13	47.38

Table S.2 — Hybrid ablation (500M, 15B tokens): GDN + SWA + Mamba2 layer orderings:

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	BoolQ acc ↑	Avg.
<i>Hybrid Ablations (500M/15B)</i>											
Gated DeltaNet + SWA + Mamba2	24.02	28.20	34.77	67.08	40.84	50.74	60.35	28.83	38.94	61.49	47.88
Gated DeltaNet + Mamba2 + SWA	23.69	26.83	36.17	67.51	41.51	51.85	61.19	29.77	38.58	53.73	47.54
Mamba2 + SWA + Gated DeltaNet	24.14	25.21	36.79	64.96	41.18	52.01	60.90	30.03	38.07	59.44	47.92
Mamba2 + Gated DeltaNet + SWA	23.54	24.11	36.92	66.48	41.70	52.72	61.06	30.54	39.91	60.51	48.73

Table S.2: Ablation studies of Gated DeltaNet models. All evaluations are performed by using lm-evaluation-harness (Gao et al., 2021). All models use the Llama tokenizer and are trained on the same subset of the FineWeb-Edu dataset (Penedo et al., 2024).

GLA / Flash Linear Attention (2024)

Scale	Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_norm ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_norm ↑	Avg. ↑
<i>340M Params 15B Tokens</i>	Transformer++	28.39	42.69	31.0	63.3	34.0	50.4	44.5	24.2	41.2
	RetNet	32.33	49.19	28.6	63.5	33.5	52.5	44.5	23.4	41.0
	Mamba	28.39	39.66	30.6	65.0	35.4	50.1	46.3	23.6	41.8
	GLA	28.65	43.35	30.3	64.8	34.5	51.4	45.1	22.7	41.5
<i>1.3B Params 100B Tokens</i>	Transformer++	16.85	13.44	48.9	70.8	49.6	53.6	56.0	26.5	50.9
	RetNet	18.64	17.27	43.3	70.0	47.3	52.5	54.8	25.6	48.9
	Mamba	17.06	13.89	46.2	72.2	40.1	54.1	59.0	28.2	50.0
	GLA	17.22	14.47	46.9	71.8	49.8	53.9	57.2	26.6	51.0

Table 2: GLA Transformer results against Transformer++ (Touvron et al., 2023), RetNet (Sun et al., 2023a), and Mamba (Gu & Dao, 2023). All models are trained on the same subset of the SlimPajama dataset with the Mistral tokenizer. The 340M/1.3B models are trained for 15B/100B tokens respectively. The individual task performance is via zero-shot. We report the main results on the same set of tasks reported by Gu & Dao (2023). See Appendix D for results on other benchmarks, including 5-shot results. The last column shows the average over all benchmarks that use (normalized) accuracy as the metric.

Scale	Model	FDA	SWDE	SQUAD
<i>340M Params 15B Tokens</i>	Transformer++	21.4	42.2	22.1
	RetNet	2.9	13.3	27.6
	Mamba	2.1	12.4	23.0
	GLA	8.1	18.6	27.2
<i>1.3B Params 100B Tokens</i>	Transformer++	27.4	66.6	31.5
	RetNet	14.3	42.8	34.7
	Mamba	6.2	41.4	35.2
	GLA	19.9	50.6	42.6

Table 3: Comparison of different models in three recall-intensive tasks tested in Arora et al. (2024). Higher is better for all tasks.

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_norm ↑	Wino. acc ↑	ARC-e acc ↑	ARC-e acc_norm ↑	COPA acc ↑	OBQA acc_norm ↑	SciQA acc ↑	BoolQ acc ↑	Avg.
<i>0-shot</i>													
Transformer++ 340M	28.39	42.69	31.0	63.3	34.0	50.4	44.5	24.2	66.0	28.4	73.8	60.9	47.7
RetNet 350M	32.33	49.19	28.6	63.5	33.5	52.5	44.5	23.4	63	28.4	73.1	60.0	47.1
Mamba 350M	28.39	39.66	30.6	65.0	35.4	50.1	46.3	23.6	71.0	28.4	73.7	52.6	47.7
GLA-Transformer 340M	28.65	43.35	30.3	64.8	34.5	51.4	45.1	22.7	70.0	29.2	73.2	58.7	48.0
<i>0-shot</i>													
Transformer++ 1.3B	16.85	13.44	48.9	70.8	49.6	53.6	56.0	26.5	75.0	29.8	83.6	52.3	54.6
RetNet 1.3B	18.64	17.27	43.3	70.0	47.3	52.5	54.8	25.6	70.0	31.4	82.3	57.1	53.4
Mamba 1.3B	17.06	13.89	46.2	72.2	40.1	54.1	59.0	28.2	74.0	33.0	83.1	59.1	54.9
GLA-Transformer 1.3B	17.22	14.47	46.9	71.8	49.8	53.9	57.2	26.6	73.0	32.4	84.7	58.5	55.5
<i>5-shot</i>													
Transformer++ 1.3B	-	16.80	42.9	70.2	50.3	53.8	60.5	28.7	75.0	33.8	90.7	46.0	55.2
RetNet 1.3B	-	23.27	37.3	69.8	47.5	51.1	58.5	27.4	72.0	31.8	87.5	45.3	52.8
Mamba 1.3B	-	23.00	31.4	71.4	51.2	54.1	60.1	30.4	79.0	33.8	88.5	47.7	55.4
GLA-Transformer 1.3B	-	18.87	41.1	71.9	49.9	54.4	61.8	28.4	75.0	34.2	90.4	56.9	56.4

Table 5: Extended zero- and five-shot performance results. All models are trained on the same subset of SlimPajama dataset with Mistral tokenizer. The 340M/1.3B models are trained for 15B/100B tokens respectively. The last column shows the average of all accuracies.

RLA / Residual DeltaNet (2025)

Table 2 — Language modeling, reasoning, commonsense (1.5B, 100B tokens, SlimPajama):

Table 3 — Recall-intensive benchmarks (DROP, FDA, NQ, SQD, SWDE, TQA, NIAH):

Table 2: Language modeling, reasoning, and commonsense understanding results. We report perplexity (lower is better) and accuracy (higher is better). The **bold/underlined** numbers indicate the first/second best values of the linear attention models in each column.

Model	Wiki ppl	LAMB ppl	ARC-C acc_n	ARC-E acc	HellaSwag acc_n	LAMB acc	MMLU acc_n	PIQA acc	SIQA acc	Wino acc	Avg Acc
Transformer	17.33	19.53	30.2	55.7	49.0	44.3	31.1	70.4	37.9	53.5	46.51
RetNet	18.86	27.62	29.4	56.0	40.5	36.5	30.3	69.9	36.0	50.9	43.69
Mamba2	18.42	20.80	29.1	<u>57.4</u>	46.2	42.7	30.9	69.4	37.6	<u>51.0</u>	45.54
sGLA	17.63	18.06	30.1	56.9	46.4	44.6	30.9	70.4	36.7	50.3	45.79
GDN	<u>17.27</u>	15.76	<u>30.8</u>	54.0	46.8	44.0	<u>31.3</u>	<u>71.3</u>	<u>38.1</u>	51.9	46.03
RLA (ours)	17.35	<u>15.59</u>	30.6	56.6	48.1	<u>46.3</u>	31.0	70.7	38.5	49.7	<u>46.44</u>
RDN (ours)	16.57	14.93	32.1	58.7	<u>47.7</u>	48.7	31.6	71.7	37.7	49.5	47.20

Table 3: Accuracy on recall-intensive benchmarks. The **bold/underlined** numbers indicate the first/second best values of the linear attention models in each column.

Model	DROP	FDA	NQ	SQD	SWDE	TQA	NIAH	Avg
Transformer	26.9	63.0	29.8	35.7	68.7	43.6	70.5	48.31
RetNet	26.2	35.3	20.8	31.5	44.1	39.9	65.7	37.64
Mamba2	26.2	41.2	23.6	33.0	<u>63.4</u>	<u>43.2</u>	67.2	42.54
sGLA	25.7	<u>51.8</u>	24.8	31.6	<u>63.4</u>	41.5	76.6	45.06
GDN	25.9	47.4	26.8	32.4	63.3	43.5	75.7	44.99
RLA (ours)	<u>26.5</u>	51.5	25.2	33.6	64.4	42.3	83.6	<u>46.73</u>
RDN (ours)	27.8	57.5	<u>26.3</u>	<u>32.5</u>	<u>63.4</u>	43.1	<u>79.2</u>	47.11

Table 4 — Ablation: residual fitting (RLA vs RLA w/o fitting, 50B tokens):

Residual Fitting To validate the importance of accumulating past errors, we test a variant that uses a simpler, predefined correction term. In this ablation, we replace our persistent auxiliary state, R_t , with a stateless correction derived only from the current residual, $R_t = (v_t - S_{t-1}k_t)k_t^\top$. As demonstrated in Table 4, the variant lacking explicit residual fitting underperforms our full method. Although this ablated variant maintains competitive performance on some benchmarks, it exhibits a substantial increase in perplexity on both the training and evaluation sets. This performance drop extends to specialized domains, with a substantial degradation in its math and code abilities, as measured by perplexity on GSM8k (Cobbe et al., 2021) and HumanEval (Chen et al., 2021). This demonstrates the critical role of the auxiliary state in accumulating past residuals to refine the model’s output effectively.

Table 4: Ablation study of the residual fitting process, comparing training loss and perplexity across various datasets. All models were pretrained for 50B tokens with the same hyperparameters, and the best results are shown in **bold**.

	Training loss	WikiText ppl	LAMB ppl	GSM8k ppl	HumanEval ppl
RLA	2.22	18.76	23.44	3.92	9.61
RLA w/o fitting	2.26	20.19	22.50	6.85	16.23

Table 5 — Ablation: gated output combination methods:

Gated Output Combination We conducted an ablation study to analyze the effect of the output combination formula. This involved comparing our full model, which uses a gated combination of the base prediction and the error correction ($o_t = \alpha_t S_{t-1}q_t + \gamma_t R_t q_t$), against a variant using simple addition ($o_t = S_{t-1}q_t + R_t q_t$). As shown in Table 5, removing the gate causes a slight performance drop for RDN but a slight increase for RLA. This outcome indicates that the core benefit is derived from the residual fitting process on the auxiliary state R_t , rather than the specific weight used to integrate the correction term.

Table 5: Ablation study of different output combination methods. All models share the same 50B tokens pretraining and hyperparameters, with the best results for each method shown in **bold**.

	Output Combination	ARC-C acc_n	ARC-E acc	HellaSwag acc_n	LAMB acc	MMLU acc_n	PIQA acc	SIQA acc	Wino acc	Avg
RLA	$\alpha_t S_{t-1}q_t + \gamma_t R_t q_t$	28.5	53.6	42.0	36.8	29.8	68.6	38.1	49.2	43.30
	$S_{t-1}q_t + R_t q_t$	28.9	55.2	41.5	37.0	30.1	68.9	37.3	51.6	43.81
RDN	$\alpha_t S_{t-1}q_t + \gamma_t R_t q_t$	29.8	55.2	42.5	39.9	30.4	69.1	40.4	49.9	44.65
	$S_{t-1}q_t + R_t q_t$	27.8	56.3	41.9	39.7	29.4	70.0	37.7	48.9	43.96

Table 6 — Model configuration; Table 7 — Training hyperparameters:

We evaluate several model architectures, with full specifications detailed in Table 6. Our comparison includes a standard Transformer and Transformers with pure linear attention. For a fair comparison, all models are trained using an identical set of hyperparameters, which are listed in Table 7. We initialize the model weights from a normal distribution with a constant standard deviation and use the AdamW optimizer with a cosine learning rate schedule for training.



Preprint

Table 6: Model Configuration

Property	Transformer Models	Linear Attention Models
Total Params	1.51B	1.55B
Hidden Size	2048	2048
Intermediate Size	8192	8192
Attention Heads	32	16
GQA Groups	4	16
Head Dimension	128	128
Softmax Attention Layers	16	0
Linear Attention Layers	0	16

Table 7: Training hyperparameters

Peak LR	Min LR	Batch Size	Warmup Tokens	Total Tokens	Weight Decay	Gradient Clip	Initialization Std
3e-4	3e-5	4M	0.5B	100B	0.1	1.0	0.006

Kimi Linear (2025)

Figure 1 — Performance vs decoding acceleration (MMLU-Pro, RULER) and TPOT vs decoding length:

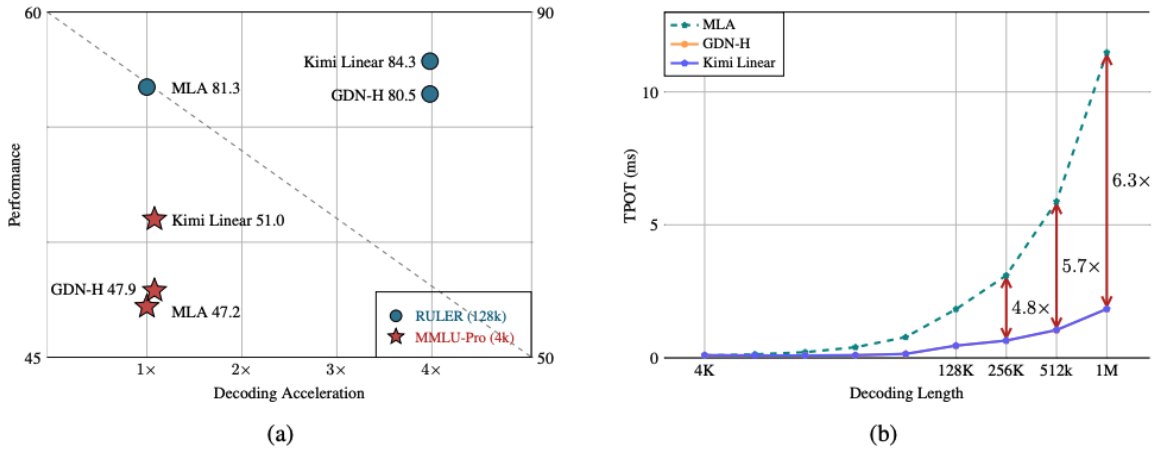


Figure 1: (a) Performance vs. acceleration. With strict fair comparisons with 1.4T training tokens, on MMLU-Pro (4k context length, red stars), Kimi Linear leads performance (51.0) at similar speed. On RULER (128k context length, blue circles), it is Pareto-optimal, achieving top performance (84.3) and 3.98 $\times$ acceleration. (b) Time per output token (TPOT) vs. decoding length. Kimi Linear (blue line) maintains a low TPOT, matching GDN-H and outperforming MLA at long sequences. This enables larger batches, yielding a 6.3 $\times$ faster TPOT (1.84ms vs. 11.48ms) than MLA at 1M tokens.

Figure 4 — Synthetic tasks: palindrome, MQAR, stack (KDA vs GDN vs Mamba2):

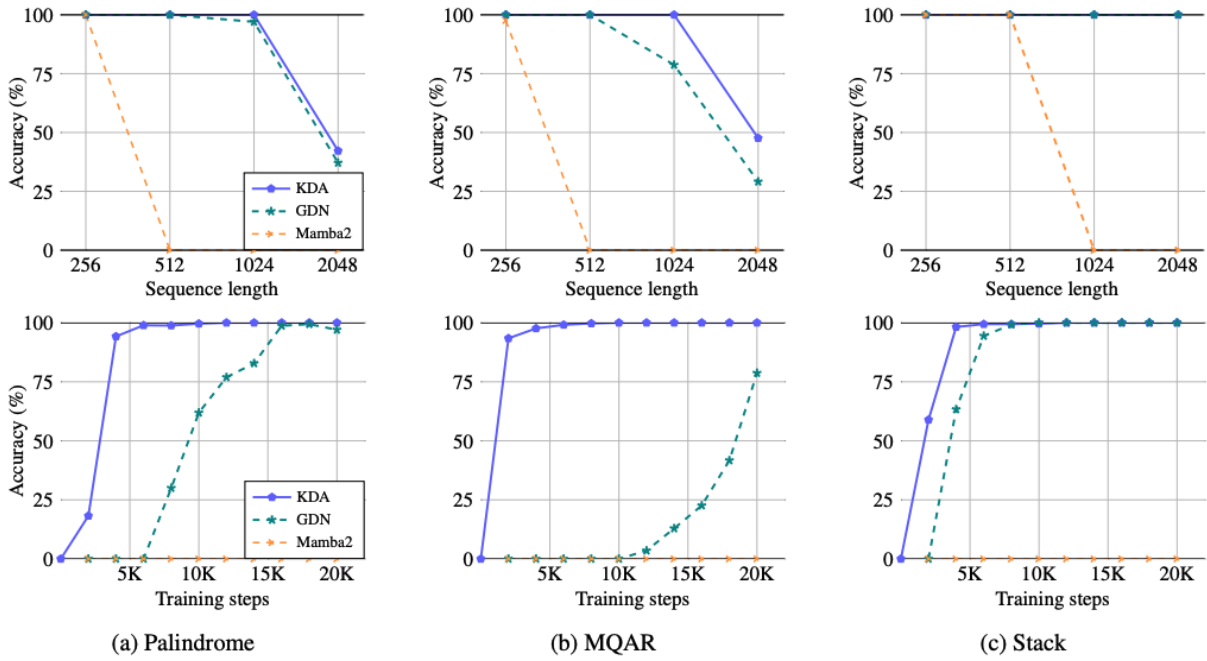


Figure 4: Results on synthetic tasks: palindrome, multi query associative recall, and the state tracking.

Table 1 — Ablation: KDA-to-MLA hybrid ratio and component ablations:

Table 1: Ablation study on the hybrid ratio of KDA to MLA attention and other key components. We list the training and validation perplexities (lower is better) for comparison. The best-performing model, used in our final experiments, is highlighted in gray.

	Training PPL ($\downarrow$)	Validation PPL ($\downarrow$)
Hybrid ratio	3:1	5.65
	0:1	5.77
	1:1	5.66
	7:1	5.70
	15:1	5.82
w/o output gate	9.25	5.67
w/swish output gate	9.43	5.81
w/o convolution layer	9.29	5.70

Table 2 — Scaling law configs and fitted scaling curves (MLA vs Kimi Linear):

# Act. Params. [†]	Head	Layer	Hidden	Tokens	lr	batch size [‡]
653M	16	16	1216	38.8B	2.006×10^{-3}	336
878M	18	18	1376	59.8B	1.790×10^{-3}	432
1.1B	20	20	1536	85.2B	1.617×10^{-3}	512
1.4B	22	22	1632	102.5B	1.486×10^{-3}	576
1.7B	24	24	1776	128.0B	1.371×10^{-3}	640

[†] Denotes the number of activated parameters in our MoE models, excluding embeddings.

[‡] All models were trained with a context length of 4,096.

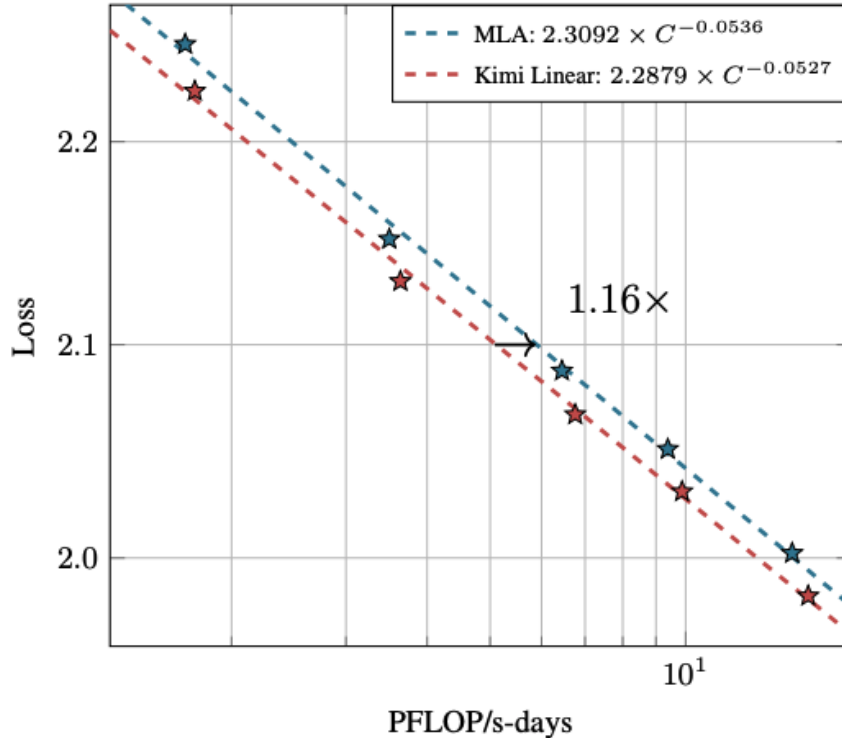


Figure 5: The fitted scaling law curves for MLA and Kimi Linear.

Evaluation benchmarks and configurations (Language, Math, Code, Chinese):

Evaluation Benchmarks Our evaluation encompasses three primary categories of benchmarks, each designed to assess distinct capabilities of the model:

- **Language Understanding and Reasoning:** Hellaswag [121], ARC-Challenge [14], Winogrande [83], MMLU [36], TriviaQA [47], MMLU-Redux [26], MMLU-Pro [103], GPQA-Diamond [82], BBH [94], and [105].
- **Code Generation:** LiveCodeBench v6 ⁴[44], EvalPlus [60].
- **Math & Reasoning:** AIME 2025, MATH 500, HMMT 2025, PolyMath-en.
- **Long-context:** MRCC ⁵, RULER [38], Frames [52], HELMET-ICL [118], RepoQA [61], Long Code Arena [13] and LongBench v2 [6].
- **Chinese Language Understanding and Reasoning:** C-Eval [43], and CMMLU [55].

Evaluation Configurations All models are evaluated using temperature 1.0. For benchmarks with high variance, we report the score of Avg@ k . For base model, We employ perplexity-based evaluation for MMLU, MMLU-Redux, GPQA-Diamond, and C-Eval. Otherwise, generation-based evaluation is adopted. To mitigate the high variance inherent to GPQA-Diamond, we report the mean score across eight independent runs. All evaluations are conducted using our internal framework derived from LM-Harness-Evaluation [10], ensuring consistent settings across all models.

Table 3 — Pretrain performance (MLA vs GDN-H vs Kimi Linear at 1.4T tokens):

Table 4 — Post-SFT instruction-tuned performance:

Table 3: Performance comparison of Kimi Linear with the full-attention MLA baseline and the hybrid GDN baseline, all after the same pretraining recipe. Kimi Linear consistently outperforms both MLA and GDN-H on short-context pretrain evaluations. Best per-column results are **bolded**.

	Type Base	MLA	GDN-H	Kimi Linear
	Trained Tokens	1.4T	1.4T	1.4T
<i>General</i>	HellaSwag	81.7	82.2	82.9
	ARC-challenge	64.6	66.5	67.3
	Winogrande	78.1	77.9	78.6
	BBH	71.6	70.6	72.9
	MMLU	71.6	72.2	73.8
	MMLU-Pro	47.2	47.9	51.0
	TriviaQA	68.9	70.1	71.7
<i>Math & Code</i>	GSM8K	83.7	81.7	83.9
	MATH	54.7	54.1	54.7
	EvalPlus	59.5	63.1	60.2
	CRUXEval-I-cot	51.6	56.0	56.6
	CRUXEval-O-cot	61.5	58.1	62.0
<i>Chinese</i>	CEval	79.3	79.1	79.5
	CMMLU	79.5	80.7	80.8

Table 4: Performance comparison of Kimi Linear with the full-attention MLA baseline and the hybrid GDN baseline, all using the same SFT recipe after pretraining. Kimi Linear consistently outperforms both MLA and GDN-H on short-context instruction-tuned benchmarks. Best per-column results are **bolded**.

	Type Instruct	MLA	GDN-H	Kimi Linear
	Trained Tokens	1.4T	1.4T	1.4T
<i>General</i>	BBH	68.2	68.5	69.4
	MMLU	75.7	75.6	77.0
	MMLU-Pro	65.7	64.8	67.4
	MMLU-Redux	79.2	78.7	80.3
	GPQA-Diamond (Avg@8)	57.1	58.6	62.1
	LiveBench (Pass@1)	45.7	46.4	45.2
<i>Math & Code</i>	AIME 2025 (Avg@64)	20.6	21.1	21.3
	MATH500 (Acc.)	80.8	83.0	81.2
	HMMT 2025 (Avg@32)	11.3	11.3	12.5
	PolyMath-en (Avg@4)	41.3	41.5	43.6
	LiveCodeBench v6 (Pass@1)	25.1	25.4	26.0
	EvalPlus	62.6	62.5	61.0

Table 5 — Long-context benchmarks (RULER, MRCL, HELMET-ICL, LongBench V2, Frames, RepoQA, Long Code Arena):

Table 5: Comparisons of Kimi Linear with MLA, GDN-H, and Kimi Linear (RoPE) across long-context benchmarks. The last column reports the overall average ($\uparrow$). All models is trained on 1.4T tokens. Best per-column results are **bolded**.

	RULER	MRCR	HELMET-ICL	LongBench V2	Frames	RepoQA	Long Code Arena		Avg.
							Lib	Commit	
MLA	81.3	22.6	88.0	36.1	60.5	63.0	32.8	33.2	52.2
GDN-H	80.5	23.9	85.5	32.6	58.7	63.0	34.7	30.5	51.2
Kimi Linear (RoPE)	78.8	22.0	88.0	35.4	59.9	66.5	31.3	32.5	51.8
Kimi Linear	84.3	29.6	90.0	35.0	58.8	68.5	37.1	32.7	54.5

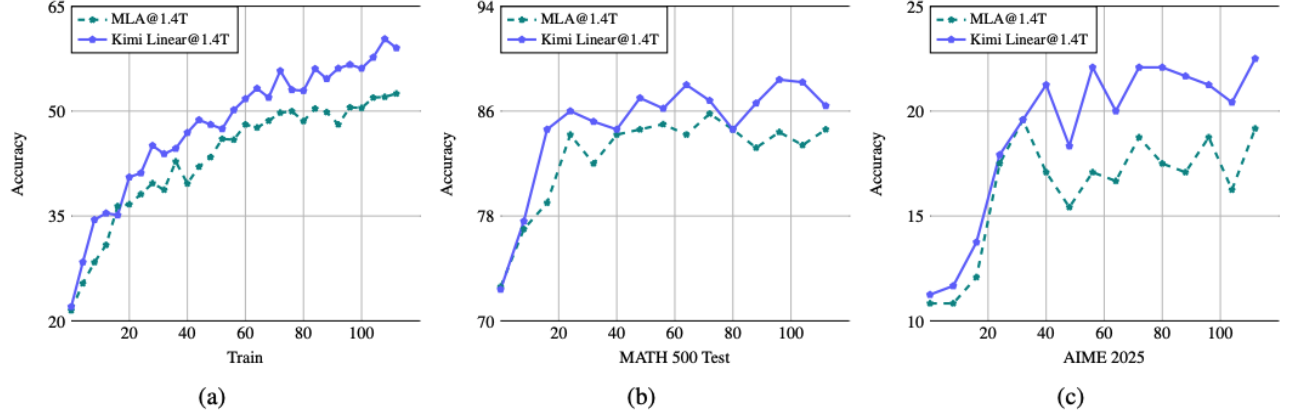


Figure 6: The training and test accuracy curves for Kimi Linear@1.4T and MLA@1.4T during Math RL training. Kimi Linear consistently outperforms the full attention baseline by a sizable margin during the whole RL process.

Tables 8–9 — Kimi Linear Base and Instruct vs Moonlight across diverse tasks:

Following Moonlight, we also trained Kimi Linear with an extended 5.7T token dataset to demonstrate its effectiveness. With $3\times$ sparsity and a new attention architecture design, Kimi Linear consistently outperforms Moonlight across nearly all benchmarks, underscoring the efficacy of the new architecture. The results are shown in Table 8 for base model and Table 9 for instruction tuned model. Moonlight-Instruct was not evaluated (“-”) on tasks exceeding its 8K context limit.

Kimi Linear@5.7T obtains a score of 94.8 on RULER at 1M context length. This long context performance reinforces that Kimi Linear is a promising alternative to full-attention architectures, delivering comparable or superior results while potentially offering more efficient resource utilization.

Table 8: Performance of Kimi-Linear-Base and Moonlight-Base across diverse tasks.

	Benchmark	#Shots	Kimi-Linear-Base	Moonlight-Base
	Architecture	-	MoE	MoE
	# Activated Params	-	3B	3B
	# Total Params	-	48B	16B
	Trained Tokens	-	5.7T	5.7T
General	TriviaQA	5-shots	75.2	66.2
	SimpleQA	5-shots	10.1	5.6
	MMLU-Pro	5-shots	54.8	42.4
	MMLU-redux	5-shots	79.7	73.8
	WinoGrande	5-shots	81.5	74.6
	GPQA-Diamond (avg@8)	5-shots	40.4	35.2
Math	MATH	4-shots	58.5	45.3
	GSM8k	8-shots	86.3	77.2
	GSM8k-platinum	8-shots	89.6	79.4
	CMATH	6-shots	85.5	79.6
Code	CRUXEval-I-cot	0-shots	61.0	45.9
	CRUXEval-O-cot	0-shots	67.0	46.6
	LiveCodeBench (v6)	1-shots	20.0	14.3
	EvalPlus	-	64.9	50.3
Chinese	C-Eval	5-shots	83.3	77.6
	CSimpleQA	5-shots	53.5	34.7

Table 9: Performance of Kimi-Linear-Instruct and Moonlight-Instruct across diverse tasks.

	Benchmark	Kimi-Linear-Instruct	Moonlight-Instruct
	Architecture	MoE	MoE
	# Activated Params	3B	3B
	# Total Params	48B	16B
	Trained Tokens	5.7T	5.7T
General	RULER@128k	95.4	-
	RULER@1M	94.8	-
	GPQA-Diamond (Avg@8)	71.7	24.7
	MMLU-Redux (EM)	86.9	66.9
	MMLU-Pro (EM)	72.7	43.8
	FaithJudge (1-Hallu.)	64.2	56.0
Math	AIME 2025 (Avg@64)	58.6	-
	MATH500 (Acc.)	94.6	58.0
	HMMT 2025 (Avg@32)	44.5	-
Code	LiveCodeBench v6 (Pass@1)	45.7	11.9
	OJBench (Pass@1)	14.2	-
	Humaneval ⁺	70.9	46.3
	MBPP ⁺	72.4	56.3

Table 4 — Recommended Experiment Setup

Component	Recommendation	Justification
Baseline architecture	Hybrid Gated DeltaNet in Megatron	Only DeltaNet-family implementation already in the Gipfelsturm Megatron fork
Target size	20 layers, hidden 1024, 16 heads, FFN 2816, GQA/KV heads 4	Close to the repo's 350m preset; preserves a known-good width/head regime
FFN multiplier	$2816 / 1024 = 2.75$	Matches the repo's current mid-scale SwiGLU style
Attention mix	<code>--experimental-attention-variant gated_delta_net with --linear-attention-freq 3</code>	Repo-tested path; yields a 2x GDN + 1x SDPA hybrid pattern
Linear-attention block dims	<code>linear-key-head-dim=64 , linear-value-head-dim=64 , conv kernel 4</code>	Matches the shipped GDN functional test config
Training dataset	SlimPajama 15B subset (fast ablations) / 100B subset (full run)	Both GLA and RLA/RDN used SlimPajama; more reproducible and literature-comparable than ClimbMix or FineWeb-Edu; publicly available
Tokenizer	LLaMA-2 (32K vocab) for paper comparability; GPT-2 BPE if minimum friction needed	GDN/GLA papers used LLaMA-2; Gipfelsturm is currently wired for GPT-2 BPE
Primary optimizer	Muon	~2x compute efficiency vs AdamW at scale (Moonlight); used by Kimi Linear (MuonClip variant); modern choice for linear attention research
AdamW baseline	Run one AdamW comparison to validate Muon matches	Confirm Muon reproduces AdamW loss curves on this architecture before committing to it for all ablations
Learning rate (Muon)	~1e-3 peak (tune based on Muon scaling recommendations)	Muon typically uses higher LR than AdamW; Kimi Linear used 1.1e-3
Learning rate (AdamW)	3e-4 peak	Matches RLA/RDN and GLA baselines on SlimPajama
Weight decay	0.1	Matches GDN and RLA/RDN

Component	Recommendation	Justification
LR schedule	Cosine decay (AdamW baseline) / WSD (Muon)	Cosine matches GDN/GLA/RLA literature; WSD is the Muon-native schedule
Warmup	0.5B tokens	Matches RLA/RDN at 100B scale; GLA used 0.5B for 340M
Min LR	3e-5 (AdamW)	Matches GLA and RLA/RDN
Global batch	4M tokens	Matches RLA/RDN; at seq len 4096 this is ~976 sequences per optimizer step
Sequence length	4096	Matches GDN/Kimi training setup and Gipfelsturm launcher
Gradient clip	1.0	Universal across all papers
Init std	0.006	Matches RLA/RDN
Precision	bf16	Already used in Megatron test configs and cluster setup
Eval cadence	WikiText-103 perplexity every checkpoint; LAMBADA accuracy; commonsense suite (PIQA, HellaSwag, WinoGrande, ARC) via <code>lm-eval</code> ; recall benchmarks (SWDE, SQD, TQA, NQ, NIAH) if time allows	WikiText + LAMBADA are cheap; commonsense is the standard DeltaNet-family comparison; recall benchmarks test the core advantage of delta-rule attention
MMLU	After checkpoint export	Script exists in repo (ModelOpt)
Fallback dataset	If SlimPajama ingest is delayed, run on <code>climbmix_small</code> first	Validates kernels, scaling, and loss curves immediately with data already present

Integration TODO List

1. Install the external `flash-linear-attention` dependency in the Alps runtime; Megatron's GDN layer hard-fails without it.
2. Add a new `300m-gdn` launcher preset in `launch.sh` with `20/1024/2816/16` and the GDN-specific args from the functional test.
3. Add a SlimPajama download + conversion script mirroring the existing ClimbMix flow (`download_*` + `convert_data.sbatch`). Target a deterministic 15B-token subset for

fast ablations and 100B for the full run.

4. Integrate the Muon optimizer into Megatron. Options: (a) use the open-source distributed Muon implementation from Moonlight, or (b) implement the Newton-Schulz orthogonalization step as a custom optimizer in Megatron's optimizer path. Run a single AdamW vs Muon comparison on the 15B subset to validate convergence.
5. Decide tokenizer path: stay on GPT-2 for fastest bring-up, or switch launcher/data prep to LLaMA-2 tokenizer for closer paper comparability.
6. Add a lightweight evaluation script path for WikiText-103 perplexity, LAMBADA, and wire the existing MMLU example as a post-checkpoint eval step. Install `lm-eval` harness for commonsense and recall benchmarks.
7. Only after baseline is stable: consider deeper work on pure DeltaNet, RDN/RLA, or Kimi-style KDA — these are all significant integrations, not small config flips.